Network Programming

Exercises 2

Computer Networks and Distributed Programming

What this pack is, and how to use it

- Twenty-six exercises on the programming block: sockets, processes, threads, synchronization, concurrent servers, multiplexing, framing and UDP.
- Three kinds. On paper: read code and say what it does, or find what is wrong. To write: a complete C program, with how to test it. To capture: run it and look at the wire.
- They are for home, numbered E2.1 to E2.26: bring the number to class if one of them resists.
- There are no solutions. A program that compiles, runs and survives the test described is the answer.
- Code on these slides leaves out error checks unless the error is the point. Yours should not.

Sockets: who needs what, and in which order

E2.1 A UDP server needs one socket, a TCP server two. Why? If the TCP server serves n clients on n different hosts at once, how many sockets does it have open?

E2.2 Take the TCP client and server and the UDP client and server you wrote for the socket lectures. Start each client before its server: what happens, and why is it different for the two protocols? Then start both, with the client using a port number different from the server's.

E2.3 In your UDP client, add `bind` to port 5432 right after `socket`. Must the server change? Which ports do the two sockets use before and after the change? Check with `ss -uan`.

Byte order

E2.4: A client on an x86 machine sends a counter; the server runs on a big-endian machine and prints what it reads.

```
1  uint32_t count = 1000;                    /* client: 0x000003E8 */
2  write(fd, &count, sizeof count);
3
4  uint32_t got;                             /* server */
5  read(fd, &got, sizeof got);               /* assume all 4 bytes arrive */
6  printf("%u\n", got);
```

(a) Write the four bytes in the order they travel. What does the server print?

(b) Fix both sides. Does your fix still work if the two machines are both x86?

(c) Which two bytes does `htons(5000)` put on the wire, in which order?

One read is not a message

E2.5: A client asks a server for a 3000-byte file and prints it.

```
1 char buf[4096];
2 write(fd, "GET file\n", 9);
3 ssize_t n = read(fd, buf, sizeof buf);
4 printf("%s", buf);
5 close(fd);
```

(a) The file arrives complete when client and server are on the same machine, and sometimes cut when they are not. Why?

(b) Name a second, independent defect in the `printf` line.

(c) Rewrite the reading part so that it works whatever the network does. How does the client know the file is over?

Find the errors (1 of 2)

E2.6: This forking server comes from an old handout. It is wrong in many places: some stop the compilation, some misbehave at run time.

```
1  #define MAX_DIM  1024
2  #define MAX_CODA 3
3
4  void main() {
5      int ds_sock, ds_sock_a;
6      struct sockaddr_in server;
7      struct sockaddr client;
8      char buff[MAX_DIM];
9
10     sock = socket(AF_INET, SOCK_STREAM, 0);
11     server.sin_family      = AF_INET;
12     server.sin_port        = 1999;
13     server.sin_addr.s_addr = INADDR_ANY;
14     bind(ds_sock, &server, sizeof(server));
```

Find the errors (2 of 2)

```
15      listen(ds_sock, MAX_CODA);
16      length = sizeof(client);
17      signal(SIGCHLD, SIG_IGN);
18      while (1) {
19          while ((ds_sock_a = accept(ds_sock, &client, &length)) == -1);
20          if (fork() == 0) {
21              do {
22                  read(ds_sock, buff, MAX_DIM);
23                  printf("client says: %s\n", buff);
24              } while (strcmp(buff, "quit") != 0);
25              write(ds_sock, "done", 10);
26              close(ds_sock_a);
27              exit(0);
28          } else close(ds_sock_a);
29      }
30  }
```

Find at least eight errors. For each, give the line, say what goes wrong, and write the fix.

Forks in a loop

E2.7: Consider this program.

```c
int main(void) {
    for (int i = 0; i < 2; i++)
        if (fork() > 0) printf("parent %d\n", i);
        else            printf("child %d\n", i);
    sleep(10);
    return 0;
}
```

(a) How many lines are printed, and how many processes exist, counting the first one? Draw the tree: who is whose child?

(b) Can you predict the order of the lines?

(c) A server does `if (fork() == 0) serve(c);` inside its accept loop, and `serve` returns when the client leaves. What happens next in that child, and how many processes end up waiting on the same listening socket?

Processes, buffers and shared files

E2.8 What does this program print, and why? What changes if both `printf` become `write(1, ...)`?

```
1 printf("A");
2 fork();
3 printf("B\n");
```

E2.9 Write a program that opens a file, forks, and has parent and child each `write` five lines (`parent 1`, `...`, `child 5`).

(a) How do the lines interleave? Does any overwrite another?

(b) Move the `open` after the `fork`: what changes, and why?

(c) A server forks after `accept`: which of the two cases is the connected socket in?

One variable, two clients

E2.10: A threaded server passes each connection to its worker like this.

```
for (;;) {
    int c = accept(ls, NULL, NULL);
    pthread_create(&tid, NULL, serve, &c);
    pthread_detach(tid);
}

void *serve(void *arg) {
    int fd = *(int *) arg;
    /* ... talk to the client on fd, then close it ... */
}
```

(a) Two clients connect a few microseconds apart. Describe an order of events in which both workers serve the second client. What happens to the first one, and to its descriptor?

(b) Fix it in two different ways.

A wait inside an if

E2.11: Two workers run this function. The main thread creates them, sleeps one second, then — holding `m` — sets `ready = 1` and calls `pthread_cond_broadcast(&c)`.

```c
int ready = 0;

void *worker(void *arg) {
    int id = (int)(long) arg;
    pthread_mutex_lock(&m);
    if (!ready)
        pthread_cond_wait(&c, &m);
    printf("thread %d in the critical section\n", id);
    ready = 0;                  /* this thread "consumes" the condition */
    pthread_mutex_unlock(&m);
    return NULL;
}
```

(a) What does the program print? (b) When each thread enters the critical section, what is the value of `ready`? (c) Fix it.

Threads that wake each other in turn

E2.12: Write a C program that creates three threads, `t1`, `t2` and `t3`, which must run in this order:

- `t1` prints `t1 starts`, then `t1 ends, wakes t2`;
- `t2` waits until `t1` wakes it, then prints `t2 starts` and `t2 ends, wakes t3`;
- `t3` waits until `t2` wakes it, then prints `t3 starts` and `t3 ends`.

Use one global `int` for the turn, one mutex and one condition variable; the main thread joins all three and destroys what it created. Test: create the threads in the order `t3`, `t2`, `t1`, and add a random `nanosleep` before each wait: the output must not change. Then answer: with a single condition variable, must you use `signal` or `broadcast`?

A client that sends and listens (1 of 2)

E2.13: A UDP client has one socket towards an echo server on `127.0.0.1:5000`. It starts `sender`, then `receiver`, and joins both.

```c
void *sender(void *arg) {
    pthread_mutex_lock(&mtx);
    sendto(fd, "A", 1, 0, (struct sockaddr *) &srv, sizeof srv);
    sendto(fd, "B", 1, 0, (struct sockaddr *) &srv, sizeof srv);
    pthread_mutex_unlock(&mtx);
    return NULL;
}
void *receiver(void *arg) {
    char buf[64];
    pthread_mutex_lock(&mtx);
    ssize_t n = recvfrom(fd, buf, sizeof buf - 1, 0, NULL, NULL);
    pthread_mutex_unlock(&mtx);
    if (n > 0) { buf[n] = '\0'; printf("got %s\n", buf); }
    return NULL;
}
```

A client that sends and listens (2 of 2)

(a) Find an order of events in which the program never ends. Which thread holds what, and what is it waiting for?

(b) What happens if the echo server is not running?

(c) What is the mutex protecting here? Does anything in these two functions need it while the I/O is in progress?

(d) Fix the design so that the client always ends, whichever thread starts first.

The parent forgot a close

E2.14: In this forking echo server, `serve` echoes lines until the client sends `quit`, then returns.

```
for (;;) {
    int c = accept(ls, NULL, NULL);
    if (fork() == 0) {
        close(ls);
        serve(c);
        close(c);
        _exit(0);
    }
    /* here the parent should close(c), and does not */
}
```

(a) A client sends `quit`. The child closes its descriptor and exits. Does the client's next `read` return 0? Why?

(b) What does `ss -tn` show on the server afterwards?

(c) After a few thousand clients, what else runs out?

Find the errors in a select loop

E2.15: A client should forward what the user types to the server, and print what the server sends, whichever comes first.

```
fd_set rset;
FD_ZERO(&rset);
FD_SET(0, &rset);
FD_SET(sock, &rset);
for (;;) {
    select(sock, &rset, NULL, NULL, NULL);
    if (FD_ISSET(0, &rset))    { /* read a line from 0, send it */ }
    if (FD_ISSET(sock, &rset)) { /* read from sock, print it */ }
}
```

Users report two symptoms: nothing from the server is ever printed, and after the first line the program behaves strangely. Find the two errors that cause them, and a third thing the loop never checks.

The server the lecture took for granted

E2.16: Write the server behind the multiplexed client of the lecture:

- it serves one client at a time, in a single thread;
- it echoes every line it receives;
- every 10 seconds it also sends a line `x`, whatever the client is doing;
- it takes the port on the command line: `./tick_server 6000`.

Use `select` with a timeout. The ten seconds must stay ten seconds even when lines keep arriving: work out how long each `select` may wait. Test: connect with `nc 127.0.0.1 6000`, type nothing, and count the `x` lines over one minute; then type a line every two seconds and count again.

A child listening to two sources

E2.17: Write a C program that creates a pipe and forks.

- The parent sends the child ten strings through the pipe, at random intervals between one and three seconds, then closes its end and waits for the child.
- The child waits on both the pipe and the keyboard with `select`. It prints `[PARENT] <text>` for what comes from the pipe and `[USER] <text>` for what the user types.
- The child ends when the parent has closed the pipe.

Test: type while the strings arrive: nothing may be lost or delayed until the next string. Then answer: how does the child learn that the parent has closed the pipe, and what would happen if the child forgot to close its own copy of the write end?

Where does the seventh client wait?

E2.18: A server's accept loop puts every new connection into a bounded queue of 4 slots; 2 worker threads take connections from the queue and serve them. When the queue is full, the accept loop blocks until a slot frees up. Serving a client takes 10 seconds. The listening socket has a backlog of 5: assume the kernel keeps at most five completed connections that nobody has accepted yet.

(a) Ten clients connect within one second. One second later, where is each of them: being served, in the queue, in the accept loop's hands, in the kernel's queue — or refused?

(b) Did the tenth client's `connect` succeed? What does it see if it sends a request now?

(c) What changes if, instead of blocking on a full queue, the accept loop closes the new connection at once?

Who receives it?

E2.19: Host A, `192.168.1.23/24`, sends UDP datagrams to port 6000. On the same LAN, B (`192.168.1.40`) has a socket bound to port 6000; C (`192.168.1.50`) has one bound to port 6000 and joined to group `239.1.2.3`. D, on `192.168.2.0/24` behind a router, is bound to 6000 and joined to the same group.

(a) For each destination address, who receives the datagram: `255.255.255.255`; `192.168.1.255`; `239.1.2.3` with TTL 1; `239.1.2.3` with TTL 8, through a router that forwards multicast.

(b) What must A set on its socket before sending to `255.255.255.255`, and what happens if it does not?

(c) Does A receive its own datagrams in any of the cases?

Framing

E2.20 TCP sockets treat data as a stream of bytes; UDP sockets keep message boundaries. Give one advantage and one disadvantage of an API that preserves application message boundaries.

E2.21 A protocol on top of TCP.

(a) A client calls `write(fd, "HELLO\n", 6)` and then `write(fd, "WORLD\n", 6)`; the server calls `read(fd, buf, 100)` once. List every result the `read` can return.

(b) Each message is framed as a 4-byte big-endian length followed by the body. The server reads, in hexadecimal, `00 00 00 03 61 62 63 00 00 00 00 00 00 00 02 68 69`. Split it into messages.

(c) What should the server do on reading `7F FF FF FF` as a length — and before which call?

Your server as the browser's proxy

E2.22: Write a TCP server in C that accepts one connection at a time and prints every line it receives on its standard output, with the line number. It takes the port on the command line: `./dump 8080`. Set the HTTP proxy of a browser to your host and port 8080, and open a page served over plain `http://`.

(a) How does the first line of each request differ from the GET you typed by hand in the first exercise pack?

(b) Reload the page. Does the browser send conditional requests (`If-Modified-Since:`, `If-None-Match:`)?

(c) The browser keeps waiting. What would your server have to do to satisfy it?

Pages served over `https://` show up as a single `CONNECT` line: that is all a proxy is allowed to see of them.

A UDP pinger

E2.23: Write two programs.

- `./udp_echo 7000` sends back every datagram it receives, but drops each one with probability 0.3.
- `./udp_ping <host> 7000` sends ten datagrams `ping <n> <time>`, one per second. It waits at most one second for each reply (`SO_RCVTIMEO`) and prints the RTT in milliseconds, or `lost`. At the end: minimum, average and maximum RTT, and the loss rate.

Test: on `127.0.0.1`, then between two machines, with a capture running.

(a) Is there one datagram on the wire per `sendto`?

(b) When a reply is lost, does anything travel to say so?

(c) A reply that arrives after its timeout: what does your client do with it, and how can it tell?

Reading your connections

E2.24: With a pipeline of commands, list all TCP connections in state `ESTAB` and print only the remote IP address, without the port: from `192.168.1.10:45678` it must print `192.168.1.10`. Hint: `ss -tn`, and `cut` with `:` as the delimiter.

(a) Test it while a browser is open on a few sites.

(b) Why does the simple `cut` break on IPv6 addresses, such as `[2a00:1450:4002::65]:443`? Change the pipeline so that it handles both.

Your own programs, on the wire

E2.25: Run a TCP server and client of yours on port 5000, and capture on the loopback interface with display filter `tcp.port == 5000`.

(a) Which call makes the first segment leave? Where is the server when the first three segments have gone by — already inside `accept`, or not yet there?

(b) The client writes 10 bytes in one call. How many segments carry them, and what travels back?

(c) It now writes 100 000 bytes in one call. How many segments, what is the largest payload, and how many segments go back?

(d) Right after the client exits, run `ss -tan`: which state names appear for port 5000, and on which side?

Datagrams, on the wire

E2.26: With a UDP sender and receiver of yours, and a capture running:

(a) One `sendto` of 3000 bytes, then three of 1000 bytes. How many datagrams do you see on loopback? Between two machines on the same LAN you will see something more: note what, it comes back with the network layer.

(b) The receiver calls `recvfrom` with a 100-byte buffer on a 1000-byte datagram. What does it return, and where are the other 900 bytes after the call?

(c) The sender sends 2000 datagrams as fast as it can; the receiver reads one per millisecond. How many are received, and where are the others lost? Would TCP lose them?

Environment and testing

- System: any Linux, a virtual machine included. Compile with warnings on, and threads where needed:

```
$ gcc -Wall -Wextra -pthread -o server server.c
```

- Testing: `nc` is the client of first resort for a TCP or UDP server; `ss -tan` and `ss -uan` show who is bound and connected.
- Capturing: on one machine, capture on the loopback interface: packets between your two programs never reach a cable.
- Nothing is to be handed in: a program that passes the test described in its exercise is the answer.